Reporte : 1p ejercicio 11

En este programa implementamos operaciones con matrices bidimensionales de tamaño 3x3 (multiplicación por un escalar y producto de dos matrices A y B) aplicando el paradigma de Programación Orientada a Objetos. La estructura del programa encapsula los datos de las matrices y la constante escalar dentro de la clase Matriz3x3, facilitando la modularidad y el control de los datos.

Código Completo:

```
#include <iostream>
using namespace std;

class Matriz3x3 {
private:
    static const int N = 3;
    int A[N][N];
    int B[N][N];
    int constante;

public:
    // Método para capturar las matrices A, B y la constante desde el teclado
    void capturarDatos() {
        cout << "Ingresa los elementos de la matriz A (3x3):\n";
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                cin >> A[i][j];
            }
        }

        cout << "Ingresa los elementos de la matriz B (3x3):\n";
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                cin >> B[i][j];
            }
        }

        cout << "Ingresa la constante: ";
        cin >> constante;
    }

    // Método para multiplicar la Matriz A por la constante
    void multiplicarConstante() const {
        cout << "\nMatriz A * constante:\n";
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                cout << A[i][j] * constante << "\t";
            }
            cout << endl;
        }
    }

    // Método para multiplicar la Matriz A por la Matriz B
    void multiplicarMatrices() const {
        int C[N][N] = {0};
```

```
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                for (int k = 0; k < N; k++) {
                    C[i][j] += A[i][k] * B[k][j];
                }
            }
        }

        cout << "\nMatriz A * B:\n";
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                cout << C[i][j] << "\t";
            }
            cout << endl;
        }
    }
};

int main() {
    Matriz3x3 matriz;

    matriz.capturarDatos();
    matriz.multiplicarConstante();
    matriz.multiplicarMatrices();

    return 0;
}
```

Explicación del Código:

1. Encapsulamiento de Matrices (private:)
Las matrices bidimensionales A[3][3] y B[3][3], la constante dimensional estática N (igual a 3) y la constante escalar se definen como miembros privados dentro de la clase Matriz3x3. Esto garantiza el encapsulamiento y protege la estructura interna de las matrices.

2. Captura de Datos (capturarDatos())
Este método utiliza ciclos for anidados para solicitar e ingresar secuencialmente los 9 elementos de la matriz A, los 9 elementos de la matriz B y el valor escalar constante desde la consola.

3. Multiplicación Escalar (multiplicarConstante())
Recorre cada celda (i, j) de la matriz A y multiplica su contenido por el valor constante ingresado, imprimiendo el resultado en forma de tabla mediante tabuladores (\t).

4. Producto de Matrices A x B (multiplicarMatrices())
Implementa el algoritmo clásico de multiplicación de matrices utilizando tres ciclos anidados. Inicializa una matriz resultado C[3][3] en ceros y acumula el producto del renglón i de la matriz A por la columna j de la matriz B (C[i][j] += A[i][k] * B[k][j]).

5. Función Principal (main)
Instancia un objeto de tipo Matriz3x3 llamado 'matriz' y manda a llamar sus métodos públicos en orden secuencial para capturar la información y calcular las operaciones.